updates, which ensure that a new deployment is listening for requests before shutting down the old deployment instances.

One slightly messy aspect of both listings 34.2 and 34.4 is that I used the Service Locator pattern to retrieve the typed client. This isn't ideal, but you shouldn't inject typed clients into background services directly. Typed clients are designed to be short-lived to ensure that you take advantage of the `HttpClient` handler rotation, as described in chapter 21. By contrast, background services are singletons that live for the lifetime of your application.

TIP If you wish, you can avoid the Service Locator pattern in listing 34.2 by using the factory pattern described in Steve Gordon's post titled "IHttpClientFactory Patterns: Using Typed Clients from Singleton Services": http://mng.bz/opDZ.

The need for short-lived services leads to another common question: how can you use scoped services in a background service?

34.1.2 Using scoped services in background tasks

Background services that implement `IHostedService` are created once when your application starts. That means they are by necessity singletons, as there will be only a single instance of the class.

That leads to a problem if you need to use services registered with a scoped lifetime. Any services you inject into the constructor of your singleton `IHostedService` must themselves be registered as singletons. Does that mean there's no way to use scoped dependencies in a background service?